

PROJETO A3: MODELAGEM E INTEGRAÇÃO COM BANCO DE DADOS EM JOGO JAVA

Integrantes do grupo:

Anderson da Silva Chaves Júnior

Matheus Oliveira Librelon

Marcos Godinho

Yasmin Camilly

Ellen Cristine

Matheus Santana

Documentação Técnica do Sistema "Cat Run 2D"

Para fins de organização, as imagens se encontram na seguinte URL:

https://github.com/AndersonJJR/Game/tree/master/A3_Modelagem

1. Definição de Escopo e Requisitos do Sistema

1.1. Visão Geral e Escopo

O projeto consiste no desenvolvimento do jogo "Cat Run 2D", um jogo do gênero *Endless Runner* (corrida infinita) desenvolvido em linguagem Java. O objetivo principal do jogo é controlar um personagem (gato) que deve desviar de obstáculos terrestres e aéreos enquanto percorre a maior distância possível. O sistema integra funcionalidades de persistência de dados para registrar o desempenho dos jogadores.

1.2. Usuários-alvo e Ambiente

- **Público-Alvo:** Jogadores casuais que buscam entretenimento rápido e competitivo através de recordes.
- **Ambiente de Uso:** Desktop (Windows, Linux, macOS) com suporte à JRE (Java Runtime Environment).

1.3. Requisitos Funcionais (RF)

Abaixo estão listadas as funcionalidades principais identificadas na análise do código fonte:

- **RF1 Autenticação de Jogador:** O sistema deve permitir que o usuário insira um ID numérico (PlayerID) antes de iniciar a partida.
- **RF2 Movimentação do Personagem:** O sistema deve processar entradas de teclado (Espaço/Seta Cima) para realizar a ação de pulo do personagem.

- **RF3 Geração Procedural de Obstáculos:** O sistema deve gerar inimigos terrestres (Enemy) e voadores (EnemyFlying) aleatoriamente.
- **RF4 Detecção de Colisão:** O sistema deve monitorar interceptações entre a área (hitbox) do jogador e dos obstáculos.
- **RF5 Contabilização de Desempenho:** O sistema deve calcular em tempo real a distância percorrida e o tempo de sobrevivência.
- **RF6 Estado de Game Over:** O jogo deve ser interrompido imediatamente após uma colisão, exibindo a tela de fim de jogo.
- **RF7 Persistência de Pontuação:** Ao término da partida, o sistema deve salvar automaticamente o ID do jogador, a distância e o tempo no banco de dados.
- **RF8 Consulta de Ranking:** O sistema deve ser capaz de recuperar as pontuações salvas para exibição (funcionalidade de backend getScores).
- **RF9 Reprodução de Áudio:** O sistema deve executar música de fundo em *loop* e efeitos sonoros em eventos específicos (pulo, colisão).
- **RF10 Controle de Estado de Animação:** O personagem deve alternar entre estados de animação (Correndo/Idle) conforme o estado do jogo.

1.4. Requisitos Não Funcionais (RNF)

- **RNF1 Desempenho:** O jogo deve manter uma taxa de atualização estável de 60 FPS (*Frames Per Second*) para garantir fluidez.
- **RNF2 Interface:** A interface gráfica deve ser construída utilizando a biblioteca Java Swing (JPanel, JFrame).
- **RNF3 Persistência:** A conexão com o banco de dados deve ser realizada via JDBC (*Java Database Connectivity*).
- **RNF4 Responsividade:** O sistema deve responder às entradas do usuário (inputs) em tempo real (baixa latência).

1.5. Riscos e Restrições

- **Restrições Tecnológicas:** O jogo é limitado ao ambiente Desktop e depende de drivers de áudio e vídeo compatíveis com Java.
- **Riscos:** Falha na conexão com o banco de dados local pode impedir o salvamento do recorde, embora não deva travar a execução da partida (tratamento de exceções SQL).

2. Casos de Uso

2.1. Descrição dos Casos de Uso

O diagrama de casos de uso foca nas interações do ator "Jogador" com o sistema.

UC01: Jogar Partida

- **Ator:** Jogador.

- **Fluxo Principal:**
 1. O Jogador inicia o aplicativo.
 2. O Sistema exibe a tela de Login.
 3. O Jogador informa o ID e clica em "Start".
 4. O Sistema inicia o loop do jogo (renderização e lógica).
 5. O Jogador controla o personagem desviando de obstáculos.
 6. O Jogador colide com um obstáculo.
 7. O Sistema encerra a partida (Game Over).

UC02: Salvar Pontuação (Sistema)

- **Ator:** Sistema.
 - **Pré-condição:** Ocorrer o evento de "Game Over".
 - **Fluxo Principal:**
 1. O Sistema captura os dados da sessão atual (PlayerID, Distância, Tempo).
 2. O Sistema estabelece conexão com o Banco de Dados.
 3. O Sistema executa a query de inserção (INSERT).
 4. O Sistema confirma a gravação e encerra a conexão.
 - **Fluxo de Exceção:** Caso o banco esteja indisponível, o sistema exibe erro no console, mas mantém o jogo aberto.
-

3. Diagrama de Classes

3.1. Visão Geral da Arquitetura

A arquitetura do sistema foi projetada de forma modular, segregando responsabilidades em pacotes distintos para facilitar a manutenção e escalabilidade.

O núcleo da aplicação reside no pacote `Main`, Ela encapsula o *Game Loop* (laço de repetição do jogo), sendo responsável por controlar o tempo de execução e ditar o ritmo em que as atualizações lógicas (`update`) e gráficas (`render`) ocorrem. A `GameEngine` mantém instâncias diretas dos componentes vitais: a janela de exibição (`GameWindow`), o painel de desenho (`GamePanel`) e o jogador (`Player`).

A interação visual ocorre através do `GamePanel` (que estende `JPanel`), que solicita frequentemente as coordenadas atualizadas das entidades. As entidades do jogo, representadas por `Player`, `Enemy` e `EnemyFlying`, seguem um modelo de herança derivado da superclasse abstrata `Entity`, garantindo que todos os objetos móveis compartilhem atributos essenciais (posição `X/Y`, dimensões e *hitbox*) e métodos de comportamento. O cenário é gerido separadamente pela classe `GameMap`, que fornece ao painel as posições das plataformas e obstáculos estáticos.

O controle do usuário é injetado no sistema através da classe `KeyInputs` (ouvinte de eventos de teclado), que altera o estado interno do `Player` (ex: iniciar um pulo). Quando a

lógica da GameEngine ou Player detecta uma colisão fatal, o fluxo normal é interrompido para acionar os serviços auxiliares: o AudioManager dispara os efeitos sonoros de "Game Over" e a classe utilitária DatabaseConnection é invocada estaticamente para persistir a pontuação final (distância e tempo) no banco de dados, fechando o ciclo de execução da partida.

3.2. Detalhamento da Classe Player

A classe Player, subclasse de Entity, encapsula a lógica do personagem controlável, gerenciando atributos físicos como coordenadas espaciais, velocidade e gravidade para simular a mecânica de pulo.

3.3. Detalhamento da Classe GameEngine

Atua como o núcleo controlador, gerenciando o *Game Loop* para garantir a execução a 60 FPS. Orquestra a atualização lógica (`update`) e renderização (`render`) do jogo, mantendo a sincronia entre os inputs, o estado das entidades e a exibição na tela.

3.4. Detalhamento da Classe GameWindow

Representa a janela principal da aplicação, responsável por instanciar o contêiner gráfico. Configura as propriedades de exibição do sistema operacional e garante que o `GamePanel` receba o foco necessário para capturar as entradas do teclado.

3.5. Detalhamento da Classe GamePanel

Componente gráfico (estendendo `JPanel`) que serve como a superfície de desenho do jogo. Sobrescreve o método `paintComponent` para renderizar visualmente o cenário, o jogador e os inimigos a cada quadro de atualização.

3.6. Detalhamento da Classe GameMap

Responsável pelo gerenciamento do ambiente, armazenando e renderizando as listas de plataformas e obstáculos estáticos. Define a geometria do nível com a qual o jogador interage, servindo de base visual para a corrida.

3.7. Detalhamento da Classe LoginPanel

Interface gráfica inicial que gerencia a autenticação simples do usuário. Contém os campos de entrada para o PlayerID e o botão de início, capturando os dados que serão posteriormente usados para persistir a pontuação no banco.

3.8. Detalhamento da Classe Main

Ponto de entrada da aplicação Java (`public static void main`). Sua função é disparar a execução do software, inicializando a interface gráfica (geralmente o `LoginPanel`) para que o fluxo do jogo possa começar.

3.9. Detalhamento da Classe Entity

Classe base que padroniza os atributos essenciais (coordenadas X/Y, largura, altura) para todos os objetos do jogo. Estabelece o contrato de métodos (update, render, getBounds) que garantem o polimorfismo entre os diferentes tipos de personagens e obstáculos.

3.10. Detalhamento da Classe Enemy

Representa os obstáculos terrestres que se movem em direção ao jogador. Herda de Entity e implementa lógica de movimentação horizontal constante, servindo como a ameaça padrão que deve ser evitada através do pulo.

3.11. Detalhamento da Classe EnemyFlying

Variação de inimigo que ocupa coordenadas verticais superiores (obstáculo aéreo). Adiciona complexidade à jogabilidade, obrigando o jogador a controlar a altura do pulo ou permanecer no chão para evitar a colisão aérea.

3.12. Detalhamento da Classe KeyInputs

Gerencia a interação humana através da captura de eventos de teclado (estendendo KeyAdapter). Converte pressionamentos de teclas (como Espaço e Setas) em variáveis booleanas de estado, permitindo que a GameEngine processe os comandos de movimentação de forma fluida.

3.13. Detalhamento da Classe DatabaseConnection

Classe utilitária responsável pela camada de persistência de dados via JDBC. Gerencia o ciclo de vida da conexão com o banco e executa instruções SQL para salvar registros ou recuperar o histórico de pontuações.

3.14. Detalhamento da Classe AudioManager

Controlador estático central para os recursos sonoros do jogo. Oferece métodos globais para iniciar a música de fundo, parar reproduções e disparar efeitos sonoros específicos em resposta a eventos do jogo.

3.15. Detalhamento da Classe SoundEffect

Encapsula clipes de áudio individuais, gerenciando seus fluxos de entrada (AudioInputStream). Permite o controle granular de reprodução (play/stop) e volume para sons curtos, como o efeito de pulo ou de colisão.

4. Protótipo das Telas

4.1. Tela de Login (LoginPanel)

Interface inicial onde o usuário se identifica.

- **Componentes:**
 - Campo de Texto (JTextField): Para entrada do ID numérico do jogador.
 - Botão (JButton): Rotulado "Iniciar Jogo" para iniciar a instância da GameWindow.
 - Fundo: Imagem estática ou cor sólida.

4.2. Tela de Jogo (GamePanel)

Interface principal onde a ação ocorre.

- **HUD (Heads-Up Display):** Exibe no canto superior a pontuação atual (Distância percorrida).
- **Área Central:** Renderização do cenário (plataformas), personagem principal e inimigos.

4.3. Tela de Game Over

Sobreposição exibida após a colisão.

- **Informações:** Exibe a mensagem "Game Over", a pontuação final atingida e o tempo de partida.
- **Ação:** O jogo é paralisado e os dados são enviados ao banco.

5. Modelo de Banco de Dados (DER)

Para atender ao requisito de persistência, foi modelado um banco de dados relacional simples.

5.1. Diagrama Entidade-Relacionamento

A persistência é gerenciada pela classe DatabaseConnection. O modelo conceitual baseia-se em uma tabela histórica de partidas.

Entidade: SCORE (Pontuação) Esta tabela armazena o histórico de todas as partidas jogadas.

Atributo	Tipo de Dado (SQL)	Descrição

id_pontuacao	INT (PK, Auto Increment)	Identificador único da partida.
id_jogador	INT	Identificador informado pelo usuário no login.
pontos	INT	Distância percorrida na partida (pontuação).
tempo_corrido	DOUBLE	Tempo total de sobrevivência em segundos.
data_partida	DATETIME	Data e hora da partida.

6. Implementação da Integração

A integração entre o jogo (Java Swing) e o Banco de Dados (MySQL/PostgreSQL) é realizada através da classe utilitária `DatabaseConnection`.

6.1. Detalhes da Implementação

- **Padrão de Projeto:** Foi utilizado o padrão *Singleton* (ou métodos estáticos) para gerenciar a conexão.
- **Tecnologia:** JDBC (`java.sql`).
- **Método `saveScore(int id, int distance, double time)`:**
 - Recebe os dados brutos da `GameEngine` ao final do jogo.
 - Prepara um `PreparedStatement` para evitar injeção de SQL.
 - Executa o comando `INSERT INTO scores ...`.
- **Momentos de Integração:** A chamada ao banco ocorre especificamente dentro do método `update()` da classe `GamePanel` ou `GameEngine`, condicionalmente quando a variável `gameOver` torna-se `true`.